Weighted Initialisation of Evolutionary Instrument and Pitch Detection in Polyphonic Music

Justin Dettmer[1(✉)], Igor Vatolkin[1], and Tobias Glasmachers[2]

[1] Chair for Artificial Intelligence Methodology, RWTH Aachen University, Aachen, Germany
{dettmer,vatolkin}@aim.rwth-aachen.de
[2] Institut für Neuroinformatik, Ruhr-University Bochum, Bochum, Germany
tobias.glasmachers@ini.rub.de

Abstract. Current state-of-the-art methods for instrument and pitch detection in polyphonic music often require large datasets and long training times; resources which are sparse in the field of music information retrieval, presenting a need for unsupervised alternative methods that do not require such prerequisites. We present a modification to an evolutionary algorithm for polyphonic music approximation through synthesis that uses spectral information to initialise populations with probable pitches. This algorithm can perform joint instrument and pitch detection on polyphonic music pieces without any of the aforementioned constraints. Sets of tuples of (instrument, style, pitch) are graded with a COSH distance fitness function and finally determine the algorithm's instrument and pitch labels for a given part of a music piece. Further investigation into this fitness function indicates that it tends to create false positives which may conceal the true potential of our modified approach. Regardless of that, our modification still shows significantly faster convergence speed and slightly improved pitch and instrument detection errors over the baseline algorithm on both single onset and full piece experiments.

Keywords: Evolutionary music approximation · Joint instrument and pitch recognition · Algorithm optimisation

1 Introduction

Automatic music transcription of polyphonic audio recordings is one of the most complex tasks in the field of Music Information Retrieval [2]. A robust algorithm which produces score from audio signals is extremely beneficial for many scenarios and applications: learning to play instruments, analysis of musical genres, rearrangement of music pieces for specific purposes, music recommendation, etc. Among several steps required for successful music transcription, instrument and pitch detection are maybe the two most prominent and not easy to solve. Several

C. Johnson et al. (Eds.): EvoMUSART 2024, LNCS 14633, pp. 114–129, 2024.
https://doi.org/10.1007/978-3-031-56992-0_8

instruments playing at the same time, different playing techniques, applied effects, and varying distributions of overtones make these tasks very challenging.

This work focuses on joint instrument and pitch detection in polyphonic music. Instead of supervised classification, which requires training of models based on preferably large annotated datasets, or also source separation, which is not always straightforward because of the unknown number of playing sources, we focus here on an analysis-by-synthesis approach like in [11]. This means that we try to re-create a close approximation of the input original sound by the combination of "basic" entities, which are instrument samples in our case. We employ such a method in this work through evolutionary approximation of the given piece, as originally proposed in [33]. The advantage of an evolutionary algorithm over the state-of-the-art Convolutional Neural Networks (CNNs) is that it does not requires training on large datasets, allows for simple addition or removal of instrument classes without a need for re-training, and the general approach remains the same independently of the number of playing sources.

Past work has shown that this rather new approach works well for feature generation in musical genre recognition [33] but is still lacking in its instrument and pitch detection capabilities. [12] has shown improvements after a systematic analysis of different input features and distance measures for the design of fitness evaluation functions. As another idea to improve the original method, we propose a technique to generate initial evolutionary algorithm solutions based on probabilities for each pitch to exclude unlikely pitches from the initial population. We further investigate the algorithm's capabilities in finding individuals with good fitness when approximating pieces that are composed of instrument recordings not contained in our sample library.

We test this modified algorithm on two synthetically created and thus perfectly annotated datasets: one containing single-onset examples and another consisting of complete musical pieces. The results show a significant reduction of instrument and pitch detection errors after initialisation, and smaller errors in the convergence area after a large number of generations. Further, we analyse the relationship between individual fitness and the associated errors.

2 Related Work

While earlier studies on instrument detection have applied classifiers like Gaussian mixture models or support vector machines using manually engineered audio features as input [4,25], more recent state-of-the-art approaches are based on CNNs [15,22] or audio spectrogram transformers [13] with Patchout [20]. The performance of neural networks can be further improved by means of Neural Architecture Search (NAS) [8], as applied in [10], or by knowledge distillation from an ensemble of teacher models to a student network [31].

Such supervised approaches require large annotated datasets for training; however, the existing datasets typically feature either only a small range of instruments, contain a smaller selection of musical pieces, or both: e.g., MedleyDB [3] has only 122 tracks and OpenMIC [17] contains annotations of only 20 instrument classes with many missing labels. Because annotation of large

datasets requires a lot of human effort, some works have introduced synthetically generated audio [24,30] which has also some limitations, as is does not contain "real-world" recorded music pieces. On the other hand, heavy optimisation techniques such as NAS involve a lot of computing resources and may sometimes lead to model overfitting.

Unsupervised methods like source-filter models and non-negative matrix factorisation [16] tackle polyphonic music by first dividing the input signal into separate parts that aim to isolate the instruments before applying classification models on the isolated signals, but have also their limitations, in particular for polyphonic music with a high and unknown number of playing instruments.

Pitch detection occurs in different problem contexts as well, and can be hindered by non-harmonic components of instrument sounds which were investigated in [23]. Chromagrams, or pitch profiles predict pitch classes without an exact estimation of the tone height. An improved version which removes timbre information was proposed in [28]. A representation that stores the strengths of all half-tones used in Western music is the semitone spectrum which can be extracted with the Non-Negative Least Squares (NNLS) chroma method [26].

A particular challenge for a robust pitch detection in polyphonic sounds is the task of fundamental frequency (f0) estimation [19]. For this task, deep convolutional neural networks were successfully applied in [32].

3 Background

In this section, we will formally describe the problem of joint instrument-pitch detection and explain the core concepts behind the Evolutionary Algorithm (EA) introduced in [33] upon which we propose our modifications in Sect. 4.

3.1 Joint Instrument-Pitch Detection

In order to talk about instrument and pitch detection accurately, we must first define the underlying combinatorial problem. For our purposes, a given musical piece will be divided into non-overlapping windows that span from one onset to another. An onset is a moment in time at which an instrument $i \in I$ begins playing a sound at pitch $p \in P$, where I is the set of all 51 possible instruments contained in our dataset, and P is the set of the 88 pitches commonly found on a piano claviature (a0 to c8). For any such onset-onset window, we are given a target set X_t of tuples (i, p), containing the instrument and pitch information for that onset. Additionally, some of the instruments are represented with several playing styles or different instrument bodies $s \in S_i$, e.g., piano samples come from five different styles of the MUMS dataset [6] as well as six sound bodies from the Native Instruments Komplete collection [29] (Alicias Keys, Gentleman, Giant hard, Giant soft, Grandeur, and Maverick). Our task is to find a candidate set X_c that is equal to the target set X_t.

The number of elements in X_t is not known a-priori, and may differ between individual onset-onset windows of a musical piece. With a large set of possible

instruments I, this combinatorial problem quickly reaches immense complexity, with up to $(|I| \cdot |P|)^n$ possibilities for a single onset, where $n \in \mathbb{N}$ is the assumed maximum possible number of tuples in X_t. This uncertainty introduces further complexity when working with polyphonic music. On top of this labelled information, we denote Y_{sig} as the musical audio signal of such an onset-onset window.

3.2 Evolutionary Approximation of Music

EAs are a type of search algorithm that simulate principles from evolution theory in order to iteratively explore a search space [7]. In our case, the search space is discrete and is comprised of all possible combinations of pitch-instrument tuples within the assumed maximum number of tuples n. We define an individual X_c as part of a population of size μ. In the baseline implementation, the initial population is created randomly. The individual is created by drawing from weighted probabilities for the number of tuples and uniform probabilities for the instruments and pitches per tuple. In addition, a valid instrument style $s \in S_i$ is drawn from the available styles of the instrument i.

In the evolutionary loop, every individual is graded with a fitness value, as described below in Sect. 3.3. Then, we randomly choose $\lambda \in \mathbb{N}$ individuals from the population, copy them, and apply one or multiple of the following mutations: (i) changing the instrument and style of a tuple, (ii) changing the pitch of a tuple, (iii) removing or adding a new tuple. The number of applied mutations n_{mut} is given by the equation

$$n_{mut} = \lfloor G \cdot \alpha + \beta \rfloor, \tag{1}$$

where α and β are hyperparameters (we refer to [33] for details), and G is drawn from a Gaussian distribution with mean 0 and standard deviation 1. The result is clipped in the range $[1, u_{bound}]$, where u_{bound} is also a hyperparameter that sets the maximum possible number of applied mutations.

The new generation of individuals is then also graded with fitness values and finally, the λ individuals with the worst fitness are removed from the population. Additionally, a mechanism for step size adaptation slowly lowers mutation strength in each generation by multiplying u_{bound} by a parameter ζ. Unless otherwise specified, we use the following parameters for our experiments: $\mu = 10$, $\lambda = 1$, $\alpha = 5$, $\beta = 10$, $u_{bound} = 10$, and $\zeta = 0.9954$.

3.3 Evaluating Fitness

Our fitness function was chosen after a set of experiments investigating the performance of different options in [12], where this choice performed the best. It operates on the signal that is generated when combining the instrument recordings contained in an individual's set of tuples. We simply sum the underlying signals saved in our sample library for this combination. The resulting signal X_{sig} is then compared to the target signal Y_{sig} by computing the average of

each Short-Time Fourier Transform (STFT) bin in the magnitude spectrum and then calculating the COSH distance [14] as given in Eq. 2. This distance measure can be interpreted as the symmetric variant of the Itakura-Saito divergence [18] and is used to compare signals within the realm of speech processing [1]. Our implementation of the COSH distance is

$$D_{\mathrm{COSH}}(X_{sig}, Y_{sig}) = \frac{1}{2N} \sum_{i=1}^{N} \left(\frac{x_i}{y_i} - \log \frac{x_i}{y_i} + \frac{y_i}{x_i} - \log \frac{y_i}{x_i} - 2 \right), \tag{2}$$

where x_i and y_i are indices in the averaged bins of the magnitude spectrum for X_{sig} and Y_{sig} respectively, and N is the number of STFT bins which is typically 1024.

As we want to closely approximate the target signal, our aim is to minimise this distance function. A zero distance indicates identical signals that we expect to only encounter when using target signals that were created from our sample library. Because full music pieces contain hundreds or even thousands of onsets, and individual onset approximation will require too much computing resources, the evolutionary individuals optimise the complete track at the same time. First, the distances between an individual (a candidate solution) and all onsets are estimated and sorted in ascending order. Then, the fitness is calculated as the mean distance to $\phi = 5\%$ of onsets with the smallest distance to the candidate solution. This way, it is aimed to assign a similar fitness to different individuals which may well approximate distinct parts of the piece characterised by a set of rather similar onsets (e.g., intro, verse, or chorus).

4 Initialisation with Pitch Probabilities

The baseline implementation of the algorithm initialises its population randomly from uniform probabilities. Every instrument and pitch has the same probability of being drawn. As the target signal is available to us, we can narrow down the distribution of candidate pitches by eliminating pitches that do not prominently appear in the spectrum of the signal. Through the Constant-Q Transform (CQT) [5], an alternative to the STFT that already operates in the logarithmic space, we can calculate magnitude bins for all of our 88 available pitches. For two piano notes (c4, g5), the CQT creates the spectrogram displayed in Fig. 1. One can clearly see that many pitches do not appear in the spectrogram at all, while the most prominent peaks are located around the correct pitches of c4 and g5. There are further maxima located in the octaves above the played notes, as well as the adjacent half tones.

Instead of relying on the CQT to set in stone which pitches we consider, we use it to create a probability distribution across our 88 pitches from which our algorithm will draw upon initialization. To do this, we must turn the two-dimensional result from applying the CQT across the target signal into a one-dimensional probability vector. We investigated two methods of dimensionality reduction: summation and the maximum method. In summation, we calculated the sum of all corresponding CQT frequency bins across the temporal axis. In

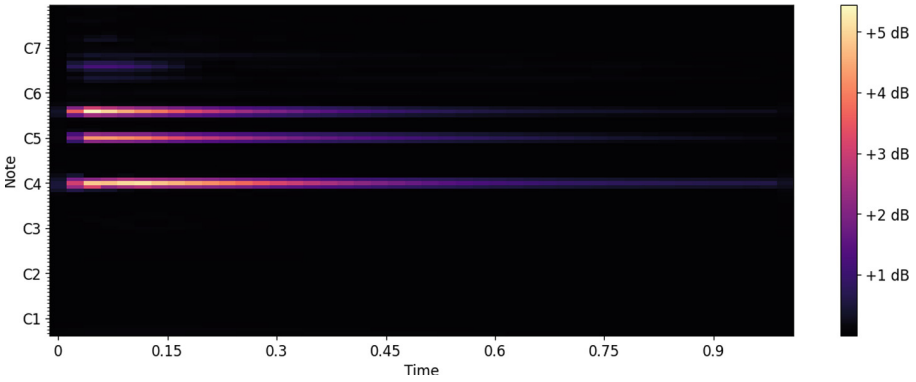

Fig. 1. A CQT magnitude spectrogram of only the harmonic elements from two piano notes (c4, g5) played together.

the maximum method, we applied the maximum function across that same axis, keeping only the largest value per frequency bin. As an additional preliminary step, we applied Harmonic Percussive Source Separation (HPSS) with median filtering as described in [9] in its librosa [27] implementation. Finally, all elements of the resulting vector are divided by the sum of elements to enforce that all probabilities sum up to exactly 1.

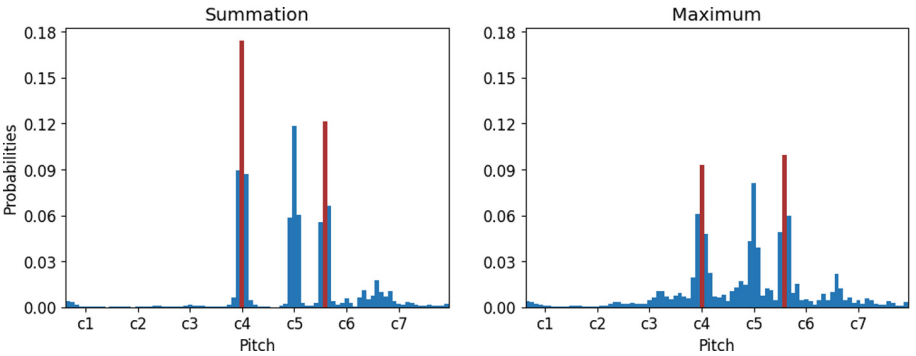

Fig. 2. Extracted pitch probabilities when summing or applying the maximum function on across the temporal axis of the CQT after applying HPSS. The correct pitches of c4 and g5 are highlighted in red. (Color figure online)

Figure 2 shows the resulting probabilities for each pitch in the example signal used for Fig. 1. We can see clear peaks around the correct pitches of c4 and g5 with both methods, however their associated probabilities are below 0.2 in both cases. Although the unlikely probabilities are barely visible in the spectrogram, their accumulation in the probability vector steals probability mass from

the desired pitches. To address this, we add an additional step in which we set all but the highest k values to zero before creating the probability vector. An example with $k = 5$ is shown in Fig. 3, where roughly 50% of the probability mass is concentrated on the two correct pitches, with the rest being distributed upon reasonable alternatives. We also observe that the summation method creates a higher peak at the lower of the two correct pitches, while the maximum method benefits the higher pitch. Qualitatively, it is hard to clearly prefer one method over the other; however, we choose to use the summation method in the remainder of this work, as it should prove to be more robust to single peaks in the CQT, and is slightly faster to calculate than the maximum function.

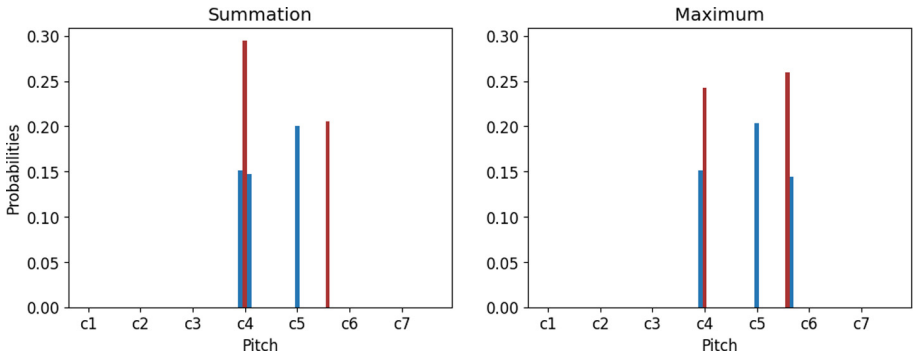

Fig. 3. Extracted pitch probabilities after keeping only the $k = 5$ largest values from calculating summation or the maximum function across the temporal axis of the CQT and applying HPSS. The correct pitches of c4 and g5 are highlighted in red. (Color figure online)

5 Experiments

In this section, we will introduce the datasets and the setup of experiments.

5.1 Evaluation Measure

All experiments use the Jaccard measure [21] for evaluation, which is defined as

$$J = 1 - \frac{|X \cap Y|}{|X \cup Y|} \tag{3}$$

for two non-empty sets X and Y. In our case, they are the candidate set X_c and target set X_t, as introduced in Sect. 3.1. A more intuitive formulation of this measure is

$$J = \frac{FP + FN}{TP + FP + FN}, \tag{4}$$

where TP is the number of true positives, FP and FN are the numbers of false positives and negatives respectively. Note that this method excludes true negatives, as most classes will be correctly labelled as negatives in our experiments.

5.2 The First Dataset: Ground Truth Search

As mentioned above, we make use of two datasets for our experiments. The first one consists of 1000 single-onset polyphonic audio mixes generated from our sample library. These examples were created the same way as individuals in our algorithm upon the first initialisation. This means that the target signal can be exactly re-created by our algorithm, giving us a good measure on whether the algorithm is able to find the global optimum reliably. In this case, the global optimum will have a fitness value of 0, as the resulting signals are identical.

For each experiment, the algorithm is run once for all 1000 targets from the dataset. At each step of the algorithm, we calculate the Jaccard errors for instrument classes (J_i), pitch classes (J_p), and joint instrument-pitch tuples (J_{ip}) for the individual with the highest fitness in that generation. The algorithm terminates prematurely if an individual with zero fitness is found to avoid unnecessary computation; otherwise, an experiment ends if the algorithm has reached its 10 000th generation.

5.3 The Second Dataset: Artificial Audio Multitracks

For our full piece approximation experiments, we use the "tiny" version of the Artificial Audio Multitracks (AAM) dataset [30]. This version of the dataset consists of 20 fully annotated pieces that were artificially created from instrument samples and then further processed in a digital audio workstation to make them sound less mechanical and more akin to human playing. Annotations for these pieces consist of the onset times, as well as the instrument-pitch tuples that are present in the audio at each onset. We use the annotated onset times to slice the piece into onset-onset windows that start at one onset and end at the subsequent onset. Each of these windows are jointly approximated as suggested in [33] by computing an individual's fitness for all windows, and then calculating the mean for only the $\phi = 5\%$ of onsets with the best fitness values, as described in Sect. 3.3.

Due to the high stochasticity of the algorithm, with random initialisation and mutations, we repeat the experiments 20 times and then average the Jaccard errors per piece across those runs. Errors are once again divided into J_i, J_p, and J_{ip} as above. Unlike the experiments in the ground truth search, we do not expect to find individuals with exactly zero fitness, because our algorithm uses a different sample library to the AAM pieces and does not apply any further processing on the generated audio signals. We run each experiment for a total of 10 000 generations and again log the Jaccard errors at the end of each generation. A deviation from the default parameters is also necessary, as the small population size of 10 is unlikely to accurately capture a wide range of onsets in the given piece. The population size μ is therefore increased to 300 for our full piece experiments.

6 Results and Discussion

In this section, we first present the results of the single onset ground truth search experiments, then the results for the full piece approximation, and finally, a short analysis of the correlation between fitness values and instrument-pitch detection errors.

6.1 Ground Truth Search

Figure 4 shows mean detection errors for the baseline algorithm across 10 000 generations. On initialisation, errors are close to 1, meaning that almost no correct instruments or pitches are found. After convergence, errors settle in around 0.18 for pitches, 0.21 for instruments, and 0.23 for the joint mean error. On closer inspection of the graphs for J_i and J_{ip}, one can see that the two graphs closely resemble each other. This resemblance indicates that the instrument and the joint detection errors often decrease together. Therefore, in these cases where the joint error decreases, it was the instrument mutation that caused the decrease, while the correct pitch was already found in the individual. This pattern is what motivates our modification to the algorithm, as it appears that the correct pitch must generally be found before finding the right instrument.

Before running experiments with the modified algorithm, we must first find a fitting cut-off value k. For that, we generate initial populations with the modified algorithm for 20 values of k and measure the pitch detection errors. The results in Fig. 5 show that larger values of k cause increased error rates and that the summation method clearly outperforms the maximum method. For all our experiments, we settled on $k = 3$ with the summation method.

Results for the modified algorithm are shown in Fig. 6. It is clear that the initial mean pitch error is drastically lower compared to the baseline algorithm, but it happens to increase in the first 80 generations. Errors after convergence are comparable to those of the baseline algorithm, however the slopes in the first few thousand generations are steeper in our modified algorithm. In cases where there is not ample time to run the algorithm for the full 10 000 generations, using our proposed method will provide better detection upon early termination.

To address the initial pitch error increase, we ran further experiments with a reduced mutation strength. The plots in Fig. 7 show the effect almost completely vanishing when setting the mutation strength to the lowest possible setting, where only a single mutation is applied to an individual. Despite the low mutation strength, final errors after convergence are not compromised. It is also necessary to note that the Jaccard error is a rather rough error metric, as it excludes many cases of correctly classified true negatives. Especially with our large set of 51 instrument labels, we consider mean errors around 0.2 to be promising for the potential of this approach.

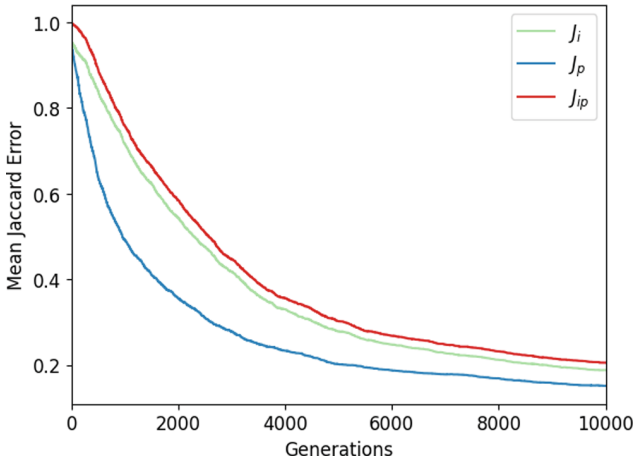

Fig. 4. Mean Jaccard errors for instrument (J_i), pitch (J_p), and joint detection (J_{ip}) across 10 000 generations of the evolutionary detection algorithm on the ground truth search dataset. The pitch detection error is the lowest, while the instrument and joint detection error closely resemble each other.

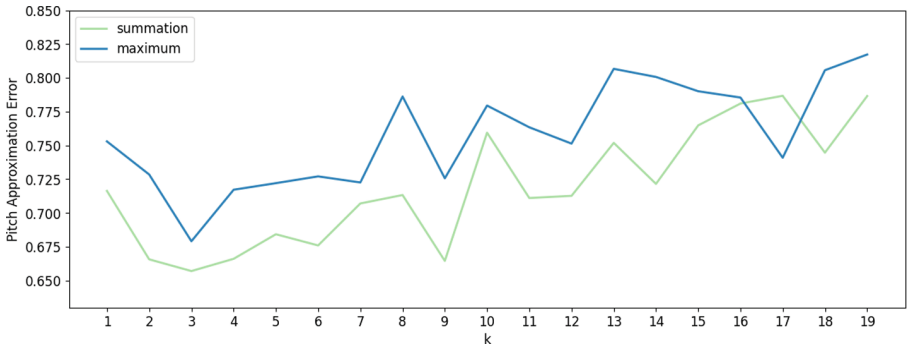

Fig. 5. Mean pitch approximation errors on the ground truth search dataset after probabilistic pitch initialisation with different values for the parameter k. The summation method outperforms the maximum method in almost all cases. Values above $k = 9$ produce increasingly higher errors.

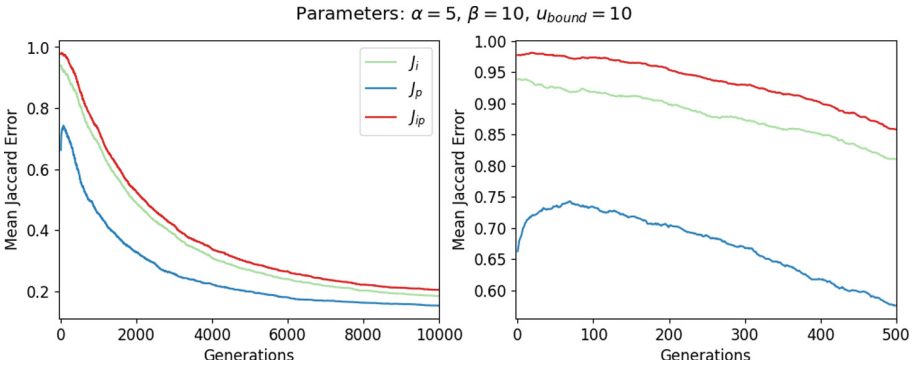

Fig. 6. Left: mean Jaccard errors for instrument (J_i), pitch (J_p), and joint detection (J_{ip}) across 10 000 generations of the evolutionary detection algorithm with probabilistic initialization on the ground truth search dataset. Right: a zoomed in view on the first 500 generations provides a closer look at the initial increase in J_p in the first 80 generations.

6.2 Artificial Audio Multitracks

Our experiments on the AAM dataset are closer to real world applications where detection tasks are often applied to entire pieces of music. As this task is significantly harder than the previous single-onset experiments, higher error rates are expected. The graphs in Fig. 8 show mean detection errors for both the baseline and the modified algorithm. We see lower initial errors for all three error classes, but surprisingly, the largest difference occurs for the instrument detection error J_i. It appears that, across an entire piece and with a much larger population, our initialisation method which only uses pitch information somehow improves the mean instrument error more than the mean pitch error. Keeping in mind the pattern noticed in the previous experiment, where pitches were generally found before instruments, this effect may be in place here too. The larger population size simply amplifies the effect to make it mostly appear in the initial population. Unlike the single-onset experiments, we can also observe a slight improvement in errors after convergence for the modified algorithm. The initial gap in J_i is strongly reduced by the time of convergence, again showing that our modification brings the largest advantage when faced with constraints in computation time. As the computation time for our pitch probabilities is negligible compared to the runtime of the algorithm itself, even its slight improvements in converged errors makes it a useful addition regardless.

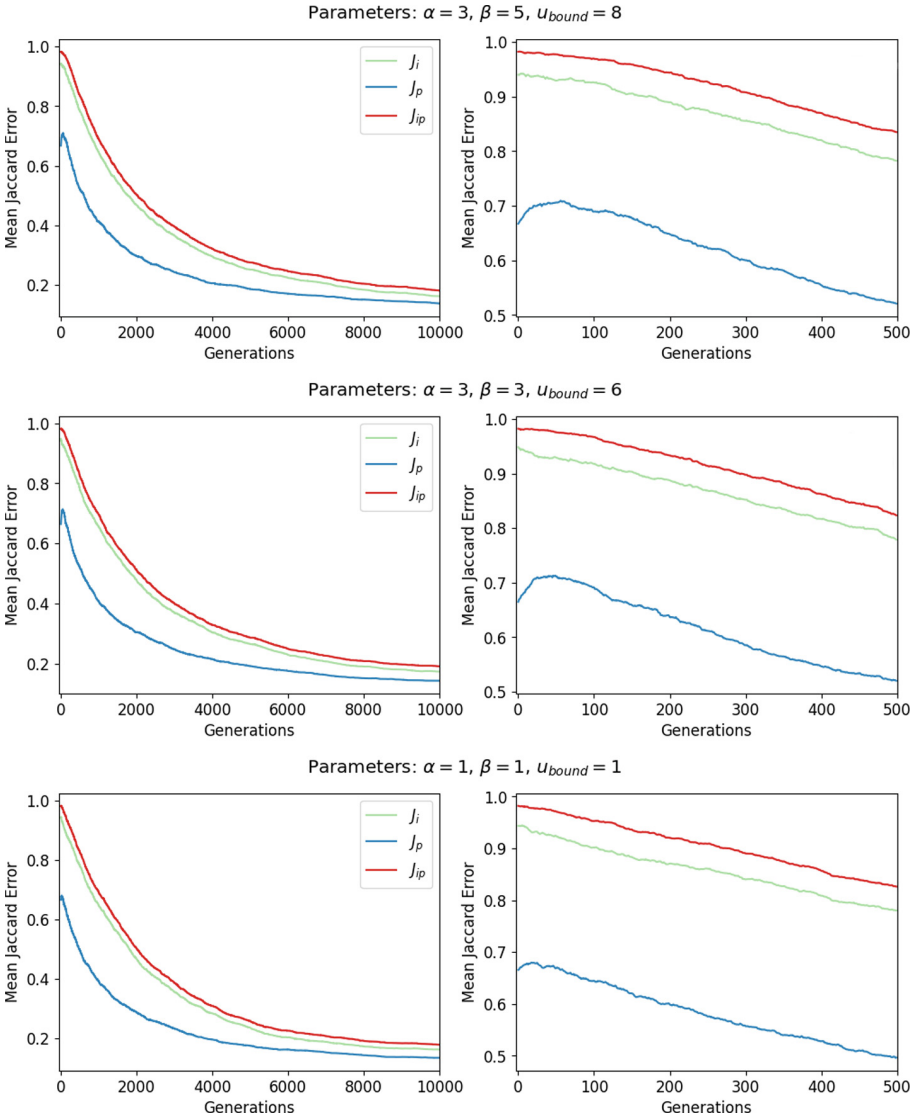

Fig. 7. Left: mean Jaccard errors for instrument (J_i), pitch (J_p), and joint detection (J_{ip}) across 10 000 generations of the evolutionary detection algorithm with probabilistic initialization on the ground truth search dataset. Mutation strength parameters introduced in Eq. 1 are varied to address the initial error increase in the first 80 generations. Right: zoomed-in views on the first 500 generations.

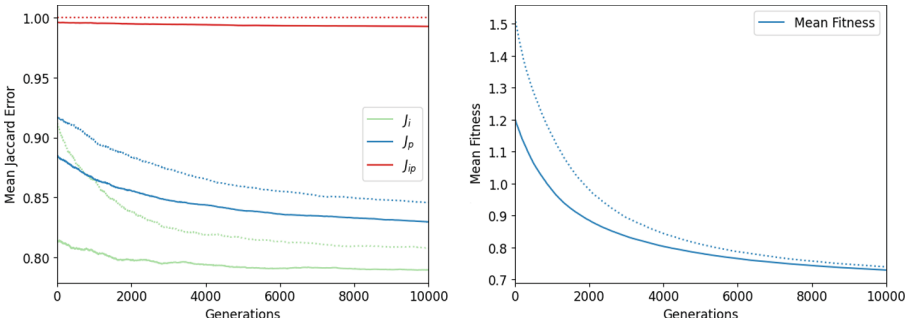

Fig. 8. Mean Jaccard errors (left) for instrument (J_i), pitch (J_p), and joint detection (J_{ip}) and mean fitness (right) across 10 000 generations of the evolutionary detection algorithm with probabilistic initialisation (solid lines) and without modification (dotted lines) on the AAM dataset. The modified algorithm produces lower mean errors than the baseline algorithm in all three error categories. Mean fitness values of the modified algorithm are lower at initialisation, but almost converge with the unmodified algorithm's fitness after 10 000 generations.

6.3 Investigating Error-Fitness Correlation

To investigate the rather high errors rates the algorithm produces on the AAM dataset, we ran a validation test in which we used a-priori knowledge from the annotated pitches and instruments to create "correct" individuals from our sample library. For each of these individuals, we ran a random search for 100 steps to find a good selection of instrument styles based on their fitness. We then calculated the fitness of these correct individuals when only compared to the onset-onset window they correctly label. This test revealed that our evolutionary approach indeed found individuals with better fitness than those with correct labels for 80.8% of onsets. The result is a high rate of false positives in both instrument and pitch detection tasks. It appears that the correlation between our magnitude-based COSH distance fitness function and the correct detection of instrument-pitch tuples is rather low, signalling a need for a more closely correlated fitness function.

Figure 8 also shows graphs for the mean fitness values in our full piece experiments, demonstrating that the fitness indeed keeps decreasing for a long time, while detection errors barely improve. One can also see that by the 10 000th generation, the mean fitness values of our modified and the baseline algorithm are close; another indication that the fitness function may indeed be a bottleneck that is hiding the true potential of our modification. Despite this, our algorithm performs far better than a random classifier (please recall a very high number of possible classes to predict), and our modification yields noticeable improvements over the baseline algorithm, as particularly demonstrated by the results in Sect. 6.1.

7 Conclusion

We presented a novel approach to initialise an algorithm for instrument and pitch detection through evolutionary synthesis. Our method uses information from the CQT spectrum to create a probability distribution from which our algorithm draws pitches for its first generation of individuals. The experiments show that this modification greatly improves mean pitch and instrument detection errors upon initialisation, but also leads to smaller errors after convergence on a full-piece detection task. The advantages of this evolutionary synthesis approach are its unsupervised nature, ability to increase or reduce the number of instrument and pitch labels without need for re-training, and its additional application in feature generation by finding spectrally close approximations of a piece that may be comprised of other instruments. Closer investigation into the detection errors made by the algorithm revealed that it indeed found individuals with good fitness values. Unfortunately, those fitness values were even better than those of individuals manually created with correct instrument-pitch tuples, suggesting a need for a modification to the current fitness function.

As an analysis on alternative single-objective fitness functions was already done in [12], we believe that a more sophisticated approach to individual fitness is required. A multi-objective fitness function that combines a wider variety of features may improve detection errors of the algorithm significantly. Among further investigations of alternative fitness functions, future work may also explore other mutation methods and extend individual's dimensions with aspects such as loudness or the addition of audio effects, e.g., reverb or compression. Other evolutionary algorithms or the integration of recombination may also be considered. For method comparison on more popular datasets, one may also reduce the size of the sample library to only the known set of instruments present in a given dataset.

In conclusion, evolutionary synthesis is a potent method for music feature calculation and joint instrument-pitch detection. Further research into potential fitness functions and more elaborate initialisation methods is required, but may lead to a promising alternative to current state-of-the-art supervised deep learning methods. To support further work on the topic, our modular code is publicly released on GitHub[1].

References

1. Bansal, M., Sircar, P.: Parametric representation of voiced speech phoneme using multicomponent am signal model. In: Proceedings of the 2018 IEEE/ACIS 17th International Conference on Computer and Information Science, ICIS, pp. 128–133 (2018)
2. Benetos, E., Dixon, S., Duan, Z., Ewert, S.: Automatic music transcription: an overview. IEEE Sig. Process. Mag. **36**(1), 20–30 (2019)

[1] https://github.com/jd-rub/EvoMUSART23-repro-code.

3. Bittner, R.M., Salamon, J., Tierney, M., Mauch, M., Cannam, C., Bello, J.P.: MedleyDB: a multitrack dataset for annotation-intensive MIR research. In: Proceedings of the 15th International Society for Music Information Retrieval Conference, ISMIR, pp. 155–160 (2014)
4. Brown, J.C., Houix, O., McAdams, S.: Feature dependence in the automatic identification of musical woodwind instruments. J. Acoust. Soc. Am. **109**(3), 1064–1072 (2001)
5. Brown, J.C., Puckette, M.S.: An efficient algorithm for the calculation of a constant Q transform. J. Acoust. Soc. Am. **92**(5), 2698–2701 (1992)
6. Eerola, T., Ferrer, R.: Instrument library (MUMS) revised. Music. Percept. **25**(3), 253–255 (2008)
7. Eiben, A.E., Smith, J.E.: Introduction to Evolutionary Computing. Natural Computing Series, 2nd edn. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-44874-8
8. Elsken, T., Metzen, J.H., Hutter, F.: Neural architecture search: a survey. J. Mach. Learn. Res. **20**, 55:1–55:21 (2019)
9. Fitzgerald, D.: Harmonic/percussive separation using median filtering. In: Proceedings of the 13th International Conference on Digital Audio Effects, DAFx, pp. 1–4 (2010)
10. Fricke, L., Vatolkin, I., Ostermann, F.: Application of neural architecture search to instrument recognition in polyphonic audio. In: Johnson, C., Rodríguez-Fernández, N., Rebelo, S.M. (eds.) EvoMUSART 2023. LNCS, vol. 13988, pp. 117–131. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-29956-8_8
11. George, E.B., Smith, M.J.: Analysis-by-synthesis/overlap-add sinusoidal modeling applied to the analysis and synthesis of musical tones. J. Audio Eng. Soc. **40**(6), 497–516 (1992)
12. Ginsel, P.: Abstandsmaße zur evolutionären Klangapproximation auf Audiodaten. Master's thesis, TU Dortmund University, Department of Computer Science (2021)
13. Gong, Y., Chung, Y.A., Glass, J.: AST: audio spectrogram transformer. In: Proceedings of the 22nd Annual Conference of the International Speech Communication Association, Interspeech, pp. 571–575 (2021). https://doi.org/10.21437/Interspeech.2021-698
14. Gray, A., Markel, J.: Distance measures for speech processing. IEEE Trans. Acoust. Speech Sig. Process. **24**(5), 380–391 (1976)
15. Han, Y., Kim, J., Lee, K.: Deep convolutional neural networks for predominant instrument recognition in polyphonic music. IEEE/ACM Trans. Audio Speech Lang. Process. **25**(1), 208–221 (2017)
16. Heittola, T., Klapuri, A., Virtanen, T.: Musical instrument recognition in polyphonic audio using source-filter model for sound separation. In: Proceedings of the 10th International Society for Music Information Retrieval Conference, ISMIR, pp. 327–332 (2009)
17. Humphrey, E., Durand, S., McFee, B.: OpenMIC-2018: an open data-set for multiple instrument recognition. In: Proceedings of the 19th International Society for Music Information Retrieval Conference, ISMIR, pp. 438–444 (2018)
18. Itakura, F.: Analysis synthesis telephony based on the maximum likelihood method. In: Reports of the 6th International Congress on Acoustics, pp. C17–20 (1968)
19. Klapuri, A.: Multiple fundamental frequency estimation based on harmonicity and spectral smoothness. IEEE Trans. Speech Audio Process. **11**(6), 804–816 (2003)

20. Koutini, K., Schlüter, J., Eghbal-zadeh, H., Widmer, G.: Efficient training of audio transformers with patchout. In: Proceedings of the 23rd Annual Conference of the International Speech Communication Association, Interspeech, pp. 2753–2757 (2022). https://doi.org/10.21437/Interspeech.2022-227
21. Levandowsky, M., Winter, D.: Distance between sets. Nature **234**(5323), 34–35 (1971)
22. Li, X., Wang, K., Soraghan, J., Ren, J.: Fusion of Hilbert-Huang transform and deep convolutional neural network for predominant musical instruments recognition. In: Romero, J., Ekárt, A., Martins, T., Correia, J. (eds.) EvoMUSART 2020. LNCS, vol. 12103, pp. 80–89. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-43859-3_6
23. Livshin, A., Rodet, X.: The significance of the non-harmonic "noise" versus the harmonic series for musical instrument recognition. In: Proceedings of the 7th International Conference on Music Information Retrieval, ISMIR, pp. 95–100 (2006)
24. Manilow, E., Wichern, G., Seetharaman, P., Le Roux, J.: Cutting music source separation some Slakh: a dataset to study the impact of training data quality and quantity. In: Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics, WASPAA (2019)
25. Marques, J., Moreno, P.J.: A study of musical instrument classification using Gaussian mixture models and support vector machines. Cambridge research laboratory technical report series CRL **4**, 143 (1999)
26. Mauch, M., Dixon, S.: Approximate note transcription for the improved identification of difficult chords. In: Proceedings of the 11th International Society for Music Information Retrieval Conference, ISMIR, pp. 135–140 (2010)
27. McFee, B., et al.: librosa: audio and music signal analysis in Python. In: Proceedings of the 14th Python in Science Conference, vol. 8, pp. 18–25 (2015)
28. Müller, M., Ewert, S.: Towards timbre-invariant audio features for harmony-based music. IEEE Trans. Audio Speech Lang. Process. **18**(3), 649–662 (2010)
29. Instruments, N.: Komplete 11 Ultimate. Native Instruments North America Inc., Los Angeles (2016)
30. Ostermann, F., Vatolkin, I., Ebeling, M.: AAM: a dataset of artificial audio multitracks for diverse music information retrieval tasks. EURASIP J. Audio Speech Music Process. **2023**(1), 13 (2023). https://doi.org/10.1186/s13636-023-00278-7
31. Schmid, F., Koutini, K., Widmer, G.: Efficient large-scale audio tagging via transformer-to-CNN knowledge distillation. In: Proceedings of the 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 1–5 (2023). https://doi.org/10.1109/ICASSP49357.2023.10096110
32. Singh, S., Wang, R., Qiu, Y.: DeepF0: end-to-end fundamental frequency estimation for music and speech signals. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, pp. 61–65 (2021)
33. Vatolkin, I.: Evolutionary approximation of instrumental texture in polyphonic audio recordings. In: Proceedings of the 2020 IEEE Congress on Evolutionary Computation, CEC, pp. 1–8 (2020)